Import Library

In [1]:
```
import pandas as pd
```

Import Data

In [2]:
```
cars = pd.read_csv('cars_data.csv')
```

Inital Insights

In [3]:
```
cars.shape #no of rows and columns
```

Out[3]:
```
(428, 15)
```

In [4]:
```
rows = cars.shape[0]
columns = cars.shape[1]
```

In [5]:
```
print("Rows : {}\nColumn : {}".format(rows,columns))
```

```
Rows : 428
Column : 15
```

In [6]:
```
cars.head()
```

Out[6]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepower
0	Acura	MDX	SUV	Asia	All	$36,945	$33,337	3.5	6.0	265.0
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	$23,820	$21,761	2.0	4.0	200.0
2	Acura	TSX 4dr	Sedan	Asia	Front	$26,990	$24,647	2.4	4.0	200.0
3	Acura	TL 4dr	Sedan	Asia	Front	$33,195	$30,299	3.2	6.0	270.0
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	$43,755	$39,014	3.5	6.0	225.0

In [7]:
```
cars.tail()
```

Out[7]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horse
423	Volvo	C70 LPT convertible 2dr	Sedan	Europe	Front	$40,565	$38,203	2.4	5.0	
424	Volvo	C70 HPT convertible 2dr	Sedan	Europe	Front	$42,565	$40,083	2.3	5.0	
425	Volvo	S80 T6 4dr	Sedan	Europe	Front	$45,210	$42,573	2.9	6.0	
426	Volvo	V40	Wagon	Europe	Front	$26,135	$24,641	1.9	4.0	
427	Volvo	XC70	Wagon	Europe	All	$35,145	$33,112	2.5	5.0	

In [8]:
```
cars.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 428 entries, 0 to 427
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   Make         428 non-null    object 
 1   Model        428 non-null    object 
 2   Type         428 non-null    object 
 3   Origin       428 non-null    object 
 4   DriveTrain   428 non-null    object 
 5   MSRP         428 non-null    object 
 6   Invoice      428 non-null    object 
 7   EngineSize   428 non-null    float64
 8   Cylinders    426 non-null    float64
 9   Horsepower   426 non-null    float64
 10  MPG_City     428 non-null    int64  
 11  MPG_Highway  428 non-null    int64  
 12  Weight       428 non-null    int64  
 13  Wheelbase    428 non-null    int64  
 14  Length       428 non-null    int64  
dtypes: float64(3), int64(5), object(7)
memory usage: 50.3+ KB
```

In [9]:
```
cars.columns #name of columns
```

Out[9]:
```
Index(['Make', 'Model', 'Type', 'Origin', 'DriveTrain', 'MSRP', 'Invoice',
       'EngineSize', 'Cylinders', 'Horsepower', 'MPG_City', 'MPG_Highway',
       'Weight', 'Wheelbase', 'Length'],
      dtype='object')
```

In [10]:
```
cars.dtypes #data types
```

Out[10]:
```
Make            object
Model           object
Type            object
Origin          object
DriveTrain      object
MSRP            object
Invoice         object
EngineSize     float64
Cylinders      float64
Horsepower     float64
MPG_City         int64
MPG_Highway      int64
Weight           int64
Wheelbase        int64
Length           int64
dtype: object
```

In [11]:
```
cars.nunique() #no of unique values
```

Out[11]:
```
Make            38
Model          425
Type             6
Origin           3
DriveTrain       3
MSRP           410
Invoice        425
EngineSize      43
Cylinders        7
Horsepower     110
MPG_City        28
MPG_Highway     33
Weight         348
Wheelbase       40
Length          67
dtype: int64
```

In [12]:
```
cars.describe()
```

Out[12]:

	EngineSize	Cylinders	Horsepower	MPG_City	MPG_Highway	Weight	Wheelbase
count	428.000000	426.000000	426.000000	428.000000	428.000000	428.000000	428.000000
mean	3.196729	5.807512	215.854460	20.060748	26.843458	3577.953271	108.154206
std	1.108595	1.558443	72.003218	5.238218	5.741201	758.983215	8.311813
min	1.300000	3.000000	73.000000	10.000000	12.000000	1850.000000	89.000000
25%	2.375000	4.000000	165.000000	17.000000	24.000000	3104.000000	103.000000
50%	3.000000	6.000000	210.000000	19.000000	26.000000	3474.500000	107.000000
75%	3.900000	6.000000	255.000000	21.250000	29.000000	3977.750000	112.000000
max	8.300000	12.000000	500.000000	60.000000	66.000000	7190.000000	144.000000

In [13]:
```
cars.isna().sum()
```

Out[13]:
```
Make           0
Model          0
Type           0
Origin         0
DriveTrain     0
MSRP           0
Invoice        0
EngineSize     0
Cylinders      2
Horsepower     2
MPG_City       0
MPG_Highway    0
Weight         0
Wheelbase      0
Length         0
dtype: int64
```

Selecting & Filtering Data

In [14]:
```
cars.iloc[0:3,0:5] #iloc[index_rows,index_column]
```

Out[14]:

	Make	Model	Type	Origin	DriveTrain
0	Acura	MDX	SUV	Asia	All
1	Acura	RSX Type S 2dr	Sedan	Asia	Front
2	Acura	TSX 4dr	Sedan	Asia	Front

In [15]:
```
cars.loc[0:5,'Make':'MSRP'] #loc[values_rows,name_column]
```

Out[15]:

	Make	Model	Type	Origin	DriveTrain	MSRP
0	Acura	MDX	SUV	Asia	All	$36,945
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	$23,820
2	Acura	TSX 4dr	Sedan	Asia	Front	$26,990
3	Acura	TL 4dr	Sedan	Asia	Front	$33,195
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	$43,755
5	Acura	3.5 RL w/Navigation 4dr	Sedan	Asia	Front	$46,100

In [16]:
```
cars[cars['MPG_City'] > 30] # Filter rows based on conditions
```

Out[16]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders
149	Honda	Civic Hybrid 4dr manual (gas/electric)	Hybrid	Asia	Front	$20,140	$18,451	1.4	4.0
150	Honda	Insight 2dr (gas/electric)	Hybrid	Asia	Front	$19,110	$17,911	2.0	3.0
154	Honda	Civic DX 2dr	Sedan	Asia	Front	$13,270	$12,175	1.7	4.0
155	Honda	Civic HX 2dr	Sedan	Asia	Front	$14,170	$12,996	1.7	4.0
156	Honda	Civic LX 4dr	Sedan	Asia	Front	$15,850	$14,531	1.7	4.0
159	Honda	Civic EX 4dr	Sedan	Asia	Front	$17,750	$16,265	1.7	4.0
352	Scion	xA 4dr hatch	Sedan	Asia	Front	$12,965	$12,340	1.5	4.0
353	Scion	xB	Wagon	Asia	Front	$14,165	$13,480	1.5	4.0
373	Toyota	Prius 4dr (gas/electric)	Hybrid	Asia	Front	$20,510	$18,926	1.5	4.0
379	Toyota	Corolla CE 4dr	Sedan	Asia	Front	$14,085	$13,065	1.8	4.0
380	Toyota	Corolla S 4dr	Sedan	Asia	Front	$15,030	$13,650	1.8	4.0
381	Toyota	Corolla LE 4dr	Sedan	Asia	Front	$15,295	$13,889	1.8	4.0
382	Toyota	Echo 2dr manual	Sedan	Asia	Front	$10,760	$10,144	1.5	4.0
383	Toyota	Echo 2dr auto	Sedan	Asia	Front	$11,560	$10,896	1.5	4.0
384	Toyota	Echo 4dr	Sedan	Asia	Front	$11,290	$10,642	1.5	4.0
404	Volkswagen	Jetta GLS TDI 4dr	Sedan	Europe	Front	$21,055	$19,638	1.9	4.0

In [17]:
```
cars[(cars['MPG_City'] > 30) & (cars['MPG_City'] < 32)]
```

Out[17]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepow
353	Scion	xB	Wagon	Asia	Front	$14,165	$13,480	1.5	4.0	108

In [18]:
```
cars[cars['Type'] == 'Sedan'] # Filter rows with string match
```

Out[18]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Hors
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	$23,820	$21,761	2.0	4.0	
2	Acura	TSX 4dr	Sedan	Asia	Front	$26,990	$24,647	2.4	4.0	
3	Acura	TL 4dr	Sedan	Asia	Front	$33,195	$30,299	3.2	6.0	
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	$43,755	$39,014	3.5	6.0	
5	Acura	3.5 RL w/Navigation 4dr	Sedan	Asia	Front	$46,100	$41,100	3.5	6.0	
...	...	...	...	...	...	...	...	...	...	
421	Volvo	S80 2.9 4dr	Sedan	Europe	Front	$37,730	$35,542	2.9	6.0	
422	Volvo	S80 2.5T 4dr	Sedan	Europe	All	$37,885	$35,688	2.5	5.0	
423	Volvo	C70 LPT convertible 2dr	Sedan	Europe	Front	$40,565	$38,203	2.4	5.0	
424	Volvo	C70 HPT convertible 2dr	Sedan	Europe	Front	$42,565	$40,083	2.3	5.0	
425	Volvo	S80 T6 4dr	Sedan	Europe	Front	$45,210	$42,573	2.9	6.0	

262 rows × 15 columns

In [19]:
```
cars[cars['Type'].str.contains('Sedan')]
```

Out[19]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Hors
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	$23,820	$21,761	2.0	4.0	
2	Acura	TSX 4dr	Sedan	Asia	Front	$26,990	$24,647	2.4	4.0	
3	Acura	TL 4dr	Sedan	Asia	Front	$33,195	$30,299	3.2	6.0	
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	$43,755	$39,014	3.5	6.0	
5	Acura	3.5 RL w/Navigation 4dr	Sedan	Asia	Front	$46,100	$41,100	3.5	6.0	
...	...	...	...	...	...	...	...	...	...	
421	Volvo	S80 2.9 4dr	Sedan	Europe	Front	$37,730	$35,542	2.9	6.0	
422	Volvo	S80 2.5T 4dr	Sedan	Europe	All	$37,885	$35,688	2.5	5.0	
423	Volvo	C70 LPT convertible 2dr	Sedan	Europe	Front	$40,565	$38,203	2.4	5.0	
424	Volvo	C70 HPT convertible 2dr	Sedan	Europe	Front	$42,565	$40,083	2.3	5.0	
425	Volvo	S80 T6 4dr	Sedan	Europe	Front	$45,210	$42,573	2.9	6.0	

262 rows × 15 columns

Drop NaN values

In [20]:
```
cars = cars.dropna(subset=['Horsepower']).reset_index(drop=True)
```

In [21]:
```
cars.isna().sum()
```

Out[21]:
```
Make          0
Model         0
Type          0
Origin        0
DriveTrain    0
MSRP          0
Invoice       0
EngineSize    0
Cylinders     2
Horsepower    0
MPG_City      0
MPG_Highway   0
Weight        0
Wheelbase     0
Length        0
dtype: int64
```

In [22]:
```
cars.tail()
```

Out[22]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horse
421	Volvo	C70 LPT convertible 2dr	Sedan	Europe	Front	$40,565	$38,203	2.4	5.0	
422	Volvo	C70 HPT convertible 2dr	Sedan	Europe	Front	$42,565	$40,083	2.3	5.0	
423	Volvo	S80 T6 4dr	Sedan	Europe	Front	$45,210	$42,573	2.9	6.0	
424	Volvo	V40	Wagon	Europe	Front	$26,135	$24,641	1.9	4.0	
425	Volvo	XC70	Wagon	Europe	All	$35,145	$33,112	2.5	5.0	

Fill NaN values in 'Cylinder' Column

In [23]:
```
cars[cars.Cylinders.isna()]
```

Out[23]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsep
245	Mazda	RX-8 4dr automatic	Sports	Asia	Rear	$25,700	$23,794	1.3	NaN	
246	Mazda	RX-8 4dr manual	Sports	Asia	Rear	$27,200	$25,179	1.3	NaN	

In [24]:
```
cars[cars.Make=='Mazda'].describe()
```

Out[24]:

	EngineSize	Cylinders	Horsepower	MPG_City	MPG_Highway	Weight	Wheelbase	
count	11.000000	9.000000	11.000000	11.000000	11.000000	11.000000	11.000000	11
mean	2.190909	4.444444	169.727273	21.454545	27.272727	2980.909091	105.090909	177
std	0.770006	0.881917	34.931622	3.587858	4.244783	435.576504	10.290331	13
min	1.300000	4.000000	130.000000	15.000000	19.000000	2387.000000	89.000000	156
25%	1.800000	4.000000	142.500000	18.000000	25.000000	2729.000000	103.500000	173
50%	2.000000	4.000000	160.000000	23.000000	28.000000	3029.000000	105.000000	178
75%	2.300000	4.000000	198.500000	24.000000	30.000000	3072.000000	109.000000	187
max	4.000000	6.000000	238.000000	26.000000	34.000000	3812.000000	126.000000	203

In [25]:
```
cars[cars.Make=='Mazda'].Cylinders.unique()
```

Out[25]:
```
array([ 4.,  6., nan])
```

In [26]:
```
cars.loc[cars['Make'] == 'Mazda', ['Cylinders']]
```

Out[26]:

	Cylinders
238	4.0
239	4.0
240	4.0
241	4.0
242	6.0
243	4.0
244	4.0
245	NaN
246	NaN
247	4.0
248	6.0

In [27]:
```
cars.loc[cars['Make'] == 'Mazda', ['Cylinders']] = cars.loc[cars['Make'] == 'Mazda'
```

In [28]:
```
cars[cars.Make=='Mazda'].isna().sum()
```

Out[28]:
```
Make          0
Model         0
Type          0
Origin        0
DriveTrain    0
MSRP          0
Invoice       0
EngineSize    0
Cylinders     0
Horsepower    0
MPG_City      0
MPG_Highway   0
Weight        0
Wheelbase     0
Length        0
dtype: int64
```

In [29]:
```
cars[cars.Make=='Mazda'].Cylinders.unique()
```

Out[29]:
```
array([4., 6.])
```

Raplace Values

In [30]:
```
cars['MSRP'] = cars['MSRP'].str.replace('$','')  # Remove $ and , in MSRP columns
cars['MSRP'] = cars['MSRP'].str.replace(',','')
```

In [31]:
```
cars['Invoice'] = cars['Invoice'].str.replace('$','')  # Remove $ and , in Invoice
cars['Invoice'] = cars['Invoice'].str.replace(',','')
```

In [32]:
```
cars.head()
```

Out[32]:

	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders	Horsepower	M
0	Acura	MDX	SUV	Asia	All	36945	33337	3.5	6.0	265.0	
1	Acura	RSX Type S 2dr	Sedan	Asia	Front	23820	21761	2.0	4.0	200.0	
2	Acura	TSX 4dr	Sedan	Asia	Front	26990	24647	2.4	4.0	200.0	
3	Acura	TL 4dr	Sedan	Asia	Front	33195	30299	3.2	6.0	270.0	
4	Acura	3.5 RL 4dr	Sedan	Asia	Front	43755	39014	3.5	6.0	225.0	

Change Data Type

In [33]:
```
cars['MSRP'] = cars['MSRP'].astype('int') # Change data type into integer
cars['Invoice'] = cars['Invoice'].astype('int')
```

In [34]:
```
cars.dtypes
```

Out[34]:
```
Make            object
Model           object
Type            object
Origin          object
DriveTrain      object
MSRP             int32
Invoice          int32
EngineSize     float64
Cylinders      float64
Horsepower     float64
MPG_City         int64
MPG_Highway      int64
Weight           int64
Wheelbase        int64
Length           int64
dtype: object
```

Modifying Data

In [35]:
```
cars.rename(columns={'EngineSize': 'Engine_Size'}, inplace=True) # Rename columns
```

In [36]:
```
cars.columns
```

Out[36]:
```
Index(['Make', 'Model', 'Type', 'Origin', 'DriveTrain', 'MSRP', 'Invoice',
       'Engine_Size', 'Cylinders', 'Horsepower', 'MPG_City', 'MPG_Highway',
       'Weight', 'Wheelbase', 'Length'],
      dtype='object')
```

In [37]:
```
cars.drop(columns=['Model'], inplace=True) # Remove columns
```

In [38]:
```
cars.columns
```

Out[38]:
```
Index(['Make', 'Type', 'Origin', 'DriveTrain', 'MSRP', 'Invoice',
       'Engine_Size', 'Cylinders', 'Horsepower', 'MPG_City', 'MPG_Highway',
       'Weight', 'Wheelbase', 'Length'],
      dtype='object')
```

In [39]:
```
cars.sort_values(by='MSRP', ascending=True) # Sort DataFrame
```

Out[39]:

	Make	Type	Origin	DriveTrain	MSRP	Invoice	Engine_Size	Cylinders	Horsepower
204	Kia	Sedan	Asia	Front	10280	9875	1.6	4.0	104.0
166	Hyundai	Sedan	Asia	Front	10539	10107	1.6	4.0	103.0
380	Toyota	Sedan	Asia	Front	10760	10144	1.5	4.0	108.0
343	Saturn	Sedan	USA	Front	10995	10319	2.2	4.0	140.0
205	Kia	Sedan	Asia	Front	11155	10705	1.6	4.0	104.0
...	...	...	...	...	...	...	...	...	...
259	Mercedes-Benz	Sedan	Europe	Rear	94820	88324	5.0	8.0	302.0
268	Mercedes-Benz	Sports	Europe	Rear	121770	113388	5.5	8.0	493.0
269	Mercedes-Benz	Sports	Europe	Rear	126670	117854	5.5	12.0	493.0
260	Mercedes-Benz	Sedan	Europe	Rear	128420	119600	5.5	12.0	493.0
332	Porsche	Sports	Europe	Rear	192465	173560	3.6	6.0	477.0

426 rows × 14 columns

Aggregation & Grouping

In [40]:
```
cars.groupby(['Origin','Type'])['MSRP'].sum()
```

Out[40]:
```
Origin  Type
Asia    Hybrid      59760
        SUV        739225
        Sedan     2139813
        Sports     552681
        Truck      163069
        Wagon      254581
Europe  SUV        483460
        Sedan     3271745
        Sports    1655970
        Wagon      454215
USA     SUV        864730
        Sedan     2307495
        Sports     407315
        Truck      435524
        Wagon      156420
Name: MSRP, dtype: int32
```

In [41]:
```
cars.groupby(['Type']).agg({'MSRP': 'sum', 'MPG_City': 'mean'}) # Multiple aggregat
```

Out[41]:

Type	MSRP	MPG_City
Hybrid	59760	55.000000
SUV	2087415	16.100000
Sedan	7719053	21.092308
Sports	2615966	18.408163
Truck	598593	16.500000
Wagon	865216	21.100000

Applying Functions

In [42]:
```
cars['MSRP']
```

Out[42]:
```
0      36945
1      23820
2      26990
3      33195
4      43755
       ...
421    40565
422    42565
423    45210
424    26135
425    35145
Name: MSRP, Length: 426, dtype: int32
```

In [43]:
```
cars['MSRP'].apply(lambda x: x*2) # Apply function to a column
```

Out[43]:
```
0      73890
1      47640
2      53980
3      66390
4      87510
       ...
421    81130
422    85130
423    90420
424    52270
425    70290
Name: MSRP, Length: 426, dtype: int64
```

In [44]:
```
cars['Type'].map({'Sedan': 1}) # Map specific values in a column
```

Out[44]:
```
0      NaN
1      1.0
2      1.0
3      1.0
4      1.0
      ...
421    1.0
422    1.0
423    1.0
424    NaN
425    NaN
Name: Type, Length: 426, dtype: float64
```

In [45]:
```
cars.apply(lambda row: row['MSRP'] - row['Invoice'], axis=1)
```

Out[45]:
```
0      3608
1      2059
2      2343
3      2896
4      4741
       ...
421    2362
422    2482
423    2637
424    1494
425    2033
Length: 426, dtype: int64
```

Merging & Joining

In [46]:
```
cars_row1 = pd.read_csv('cars_data_row1.csv')
```

In [47]:
```
cars_row1.shape
```

Out[47]:
```
(199, 15)
```

In [48]:
```
cars_row2 = pd.read_csv('cars_data_row2.csv')
```

In [49]:
```
cars_row2.shape
```

Out[49]:
```
(229, 15)
```

In [50]:
```
cars_row = pd.concat([cars_row1, cars_row2], axis=0) # Concatenate along rows (stac
```

In [51]:
```
cars_row.shape
```

Out[51]:
```
(428, 15)
```

In [52]:
```
cars_col1 = pd.read_csv('cars_data_col1.csv')
```

In [53]:
```
cars_col1.shape
```

Out[53]:
```
(428, 7)
```

In [54]:
```
cars_col2 = pd.read_csv('cars_data_col2.csv')
```

In [55]:
```
cars_col2.shape
```

Out[55]:
```
(428, 8)
```

In [56]:
```
cars_col = pd.concat([cars_col1, cars_col2], axis=1) # Concatenate along columns
```

In [57]:
```
cars_col.shape
```

Out[57]:
```
(428, 15)
```

In [58]:
```
cars_data_merge1 = pd.read_csv('cars_data_merge1.csv')
```

In [59]:
```
cars_data_merge2 = pd.read_csv('cars_data_merge2.csv')
```

In [60]:
```
cars_data_merge = cars_data_merge1.merge(cars_data_merge2, on='id', how='inner') #
```

In [61]:
```
cars_data_merge
```

Out[61]:

	id	Make	Model	Type	Origin	DriveTrain	MSRP	Invoice	EngineSize	Cylinders
0	1	Acura	MDX	SUV	Asia	All	$36,945	$33,337	3.5	6.0
1	2	Acura	RSX Type S 2dr	Sedan	Asia	Front	$23,820	$21,761	2.0	4.0
2	3	Acura	TSX 4dr	Sedan	Asia	Front	$26,990	$24,647	2.4	4.0
3	4	Acura	TL 4dr	Sedan	Asia	Front	$33,195	$30,299	3.2	6.0
4	5	Acura	3.5 RL 4dr	Sedan	Asia	Front	$43,755	$39,014	3.5	6.0
...	...	...	...	...	...	...	...	...	...	...
423	424	Volvo	C70 LPT convertible 2dr	Sedan	Europe	Front	$40,565	$38,203	2.4	5.0
424	425	Volvo	C70 HPT convertible 2dr	Sedan	Europe	Front	$42,565	$40,083	2.3	5.0
425	426	Volvo	S80 T6 4dr	Sedan	Europe	Front	$45,210	$42,573	2.9	6.0
426	427	Volvo	V40	Wagon	Europe	Front	$26,135	$24,641	1.9	4.0
427	428	Volvo	XC70	Wagon	Europe	All	$35,145	$33,112	2.5	5.0

428 rows × 16 columns

Pivoting & Reshaping Data

In [62]:
```
cars.pivot(columns='Origin', values='MSRP') #Pivot table
```

Out[62]:

Origin	Asia	Europe	USA
0	36945.0	NaN	NaN
1	23820.0	NaN	NaN
2	26990.0	NaN	NaN
3	33195.0	NaN	NaN
4	43755.0	NaN	NaN
...	...	...	...
421	NaN	40565.0	NaN
422	NaN	42565.0	NaN
423	NaN	45210.0	NaN
424	NaN	26135.0	NaN
425	NaN	35145.0	NaN

426 rows × 3 columns

In [63]:
```
cars.T # Transpose
```

Out[63]:

	0	1	2	3	4	5	6	7	8	9	...	
Make	Acura	Acura	Acura	Acura	Acura	Acura	Acura	Audi	Audi	Audi	...	V
Type	SUV	Sedan	Sedan	Sedan	Sedan	Sedan	Sports	Sedan	Sedan	Sedan	...	Se
Origin	Asia	Asia	Asia	Asia	Asia	Asia	Asia	Europe	Europe	Europe	...	Eu
DriveTrain	All	Front	Front	Front	Front	Front	Rear	Front	Front	Front	...	
MSRP	36945	23820	26990	33195	43755	46100	89765	25940	35940	31840	...	3
Invoice	33337	21761	24647	30299	39014	41100	79978	23508	32506	28846	...	2
Engine_Size	3.5	2.0	2.4	3.2	3.5	3.5	3.2	1.8	1.8	3.0	...	
Cylinders	6.0	4.0	4.0	6.0	6.0	6.0	6.0	4.0	4.0	6.0	...	
Horsepower	265.0	200.0	200.0	270.0	225.0	225.0	290.0	170.0	170.0	220.0	...	2
MPG_City	17	24	22	20	18	18	17	22	23	20	...	
MPG_Highway	23	31	29	28	24	24	24	31	30	28	...	
Weight	4451	2778	3230	3575	3880	3893	3153	3252	3638	3462	...	3
Wheelbase	106	101	105	108	115	115	100	104	105	104	...	
Length	189	172	183	186	197	197	174	179	180	179	...	

14 rows × 426 columns

Creating Dataframe

In [64]:
```
cols = ['Name','Gender','DOB','Mobile_no','Email']
rows = [['Mitesh','Male','24-09-1985','9828555308','mitesh.parishkar@gmail.com']]
data = pd.DataFrame(data=rows,columns=cols)
```

In [65]:
```
data
```

Out[65]:

	Name	Gender	DOB	Mobile_no	Email
0	Mitesh	Male	24-09-1985	9828555308	mitesh.parishkar@gmail.com

In [66]:
```
data.dtypes
```

Out[66]:
```
Name         object
Gender       object
DOB          object
Mobile_no    object
Email        object
dtype: object
```

Working with Dates

In [67]:
```
import warnings
```

In [68]:
```
warnings.filterwarnings('ignore')
```

In [69]:
```
data['DOB'] = pd.to_datetime(data['DOB']) # Convert column to datetime
```

In [70]:
```
data.dtypes
```

Out[70]:
```
Name                object
Gender              object
DOB         datetime64[ns]
Mobile_no           object
Email               object
dtype: object
```

In [71]:
```
data['DOB'].dt.year # Extract year
```

Out[71]:
```
0    1985
Name: DOB, dtype: int32
```

In [72]:
```
data['DOB'].dt.month # Extract month
```

Out[72]:
```
0    9
Name: DOB, dtype: int32
```

In [73]:
```
data['DOB'].dt.day # Extract day
```

Out[73]:
```
0    24
Name: DOB, dtype: int32
```

Exporting Processed Data

In [74]:
```
data.to_csv('data.csv', index=False)
```

In [ ]:
```

```